01 / DECISION

A sensor test bench you can explain

ESP32 + NTC thermistor + Python validation + SPC

Recommended project: a small, reproducible temperature acquisition bench that converts an analog signal, logs data, detects faults, and evaluates measurement stability. Confidence: **high** as a fit for the prototype/documentation request; **medium** for the exact interview team because its role is not identified in the attached email.

Why this is the strongest practical choice

It builds on your thermistor and embedded-logging work, uses the ESP32 and bench components you have described, and gives you concrete decisions to discuss: divider sizing, ADC limitations, filtering, uncertainty, fault isolation and acceptance testing. A working, understood prototype is more persuasive than a larger unverified design.

What the interviewer requested	What to bring
Prototype	Assembled breadboard, USB cable, DMM and laptop. Physical build remains yours to complete.
Schematic and spec	Wiring-level schematic, BOM, calculations, requirements and verification matrix in this report.
Software examples you wrote	Bring original code you actually wrote as well. New package code is AI-assisted; review, modify and explain it accurately.

Evidence boundary

Completed here: design documentation, executable Python model, synthetic datasets, charts, five passing Python tests and source listings. **Not completed:** physical assembly, Arduino compilation, firmware upload, serial capture or temperature calibration. No synthetic result is a measured achievement.

The attached email says Wednesday at 3 pm and asks for project prototypes if possible. Confirm the calendar invitation for the exact date and location. This report is a bench demonstration, not flight hardware or qualified aircraft ground-support equipment.

02 / SPECIFICATION

Define success before testing

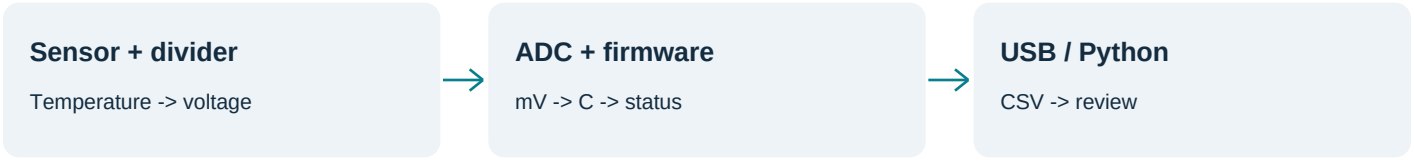
ID	Proposed requirement / acceptance criterion	Evidence status
R1	Measure 15 to 55 C with identified 10k B3950 NTC; report out-of-range beyond that interval.	Model only
R2	Target absolute error <=1.0 C at 15, 25, 35, 45 and 55 C against reference uncertainty <=0.2 C. Apply guardband.	Unverified target
R3	Fixed-condition repeatability: standard deviation <=0.15 C over 60 s after stabilization.	Not measured
R4	Nominal output 5 records/s; observe 4.5-5.5 Hz over 60 s, no malformed records.	Not measured
R5	<=100 mV LOW_FAULT; >=2900 mV HIGH_FAULT. Detect stable injected fault within 1 s.	Python logic tested
R6	HOT at >=45 C; return NORMAL at <=43 C; keep previous state in between.	Python logic tested
R7	CSV contains time, mV, temperature and state; invalid sensor temperature must not be treated as a valid measurement.	Parser authored
R8	Power from USB only; ADC signal stays within board electrical limits. No aircraft connection.	Wiring review needed

Do not call R2 achieved. Its feasibility depends on actual thermistor tolerance, supply measurement, ADC behavior and calibration. ADC averaging reduces random noise; it does not remove systematic error.

The operating band, thresholds and limits are project design choices. They are not requirements supplied by the interviewer or claims from the component manufacturer.

03 / ARCHITECTURE

From signal to evidence



Data path: one firmware row every nominal 200 ms. Each row averages 16 calibrated-mV ADC conversions. Temperature conversion uses the measured rail and resistor values; threshold logic runs before conversion at the fault rails.

Layer	Responsibilities	Failure response
Circuit	3V3 divider, series isolation resistor, ADC capacitor, common ground.	DMM check before USB logging.
Firmware	Acquire, convert, apply hysteresis, report fault/range state.	Fault rows carry blank temperature.
Python capture	Parse serial CSV, identify hardware rows, count rejected rows.	Stop on zero valid records; keep faults.
Analysis	Separate reference sweeps and steady-state repeatability; freeze SPC limits.	Flag changes; do not auto-adjust limits.

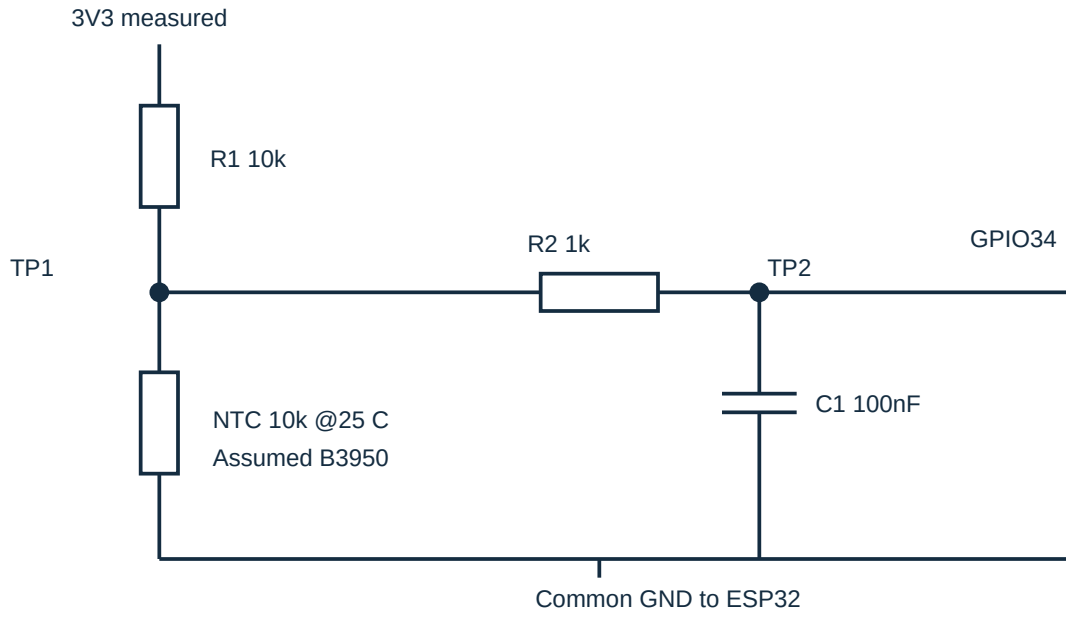
A deliberate limitation

The firmware does not control a heater, relay or aircraft system. HOT is an informational label. There is no safety-rated shutdown, redundant sensor, watchdog validation or environmental qualification. Range faults indicate an invalid reading but cannot identify every wiring failure uniquely.

04 / SCHEMATIC

Breadboard wiring, Rev A

Target assumption: original ESP32-WROOM DevKit, ADC1 input GPIO34.



USB powers the DevKit; the board 3V3 rail supplies the divider. Connect R1 from 3V3 to TP1, the thermistor from TP1 to GND, R2 from TP1 to GPIO34, and C1 from GPIO34 to GND. R2 is not a substitute for overvoltage protection.

Check	Expected before logging
Power off	Verify wiring and no accidental 3V3-to-GND short; confirm NTC part number.
Power on / DMM	Measure 3V3 and enter VS_MV. At nominal 25 C and matched 10k resistors, TP1 is half the rail.
Pinout	GPIO34 is for the original ESP32 assumption only. Check exact board pinout before use.
Fault injection	Power down before changing connections. Remove NTC for high-rail fault; bypass NTC for low-rail fault.

05 / PARTS & MATH

Understand every component

Part	Quantity / value	Reason
ESP32 DevKit	1; original WROOM assumed	ADC1 GPIO34 and USB serial
NTC	1; 10k @25 C, B3950 assumed	Identify actual tolerance and beta interval
R1	1; 10k, preferably 0.1%	Match nominal NTC for useful sensitivity
R2 / C1	1k / 100nF ceramic	Input isolation and low-pass filtering
Bench supplies	Breadboard, jumpers, USB data cable, DMM	Assembly, logging and voltage verification
Reference / substitutions	Reference thermometer; measured 10k, 6.8k and 3.3k resistors	Thermal validation and electrical-only checks

Model: R(T) = R25 exp[B(1/T - 1/298.15)], with T in kelvin.
Divider: V = Vs RNTC / (R1 + RNTC).
Inverse: RNTC = R1 V / (Vs - V); T(C) = 1/[1/298.15 + ln(RNTC/R25)/B] - 273.15.

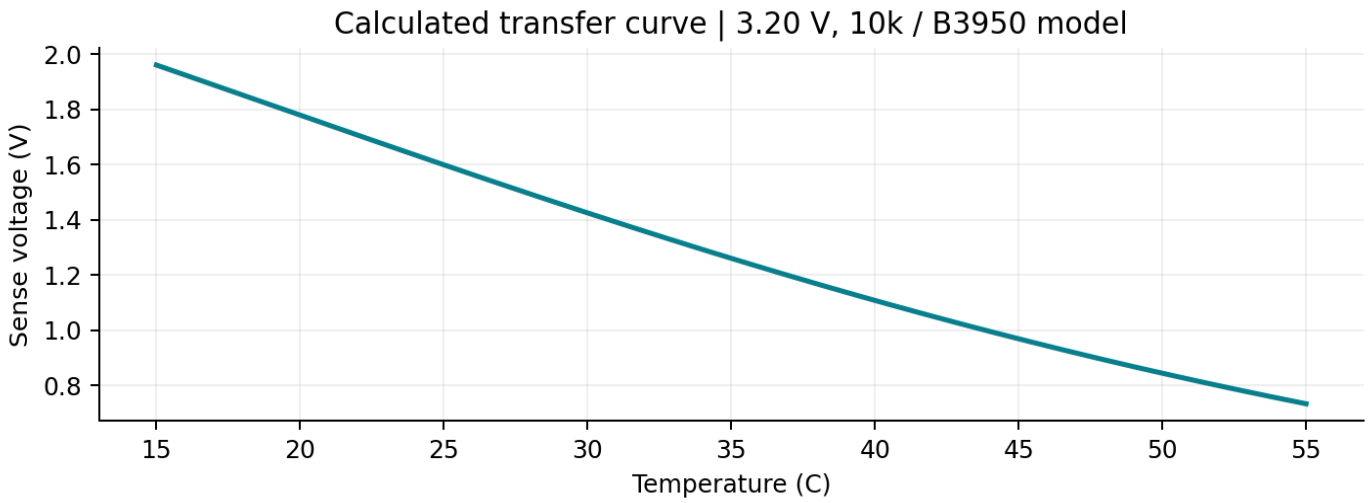
At 25 C with Vs = 3.20 V: V = 1.600 V, divider current = 160 microamps, NTC power = 0.256 mW. Self-heating rise is P / dissipation constant; that constant is unknown until the sensor is identified. It must be evaluated, not assumed negligible.

At 25 C, R1 || RNTC = 5k. Approximate filter time constant = (5k + 1k) x 100nF = 0.60 ms, cutoff about 265 Hz. This aids ADC settling and noise suppression; it is not a complete anti-alias filter. Sensor thermal response and 5 Hz reporting dominate the visible demo.

Original ESP32 documentation describes an approximate 150-3100 mV range at 11 dB attenuation. The intended 15-55 C band is inside that range; near-rail faults are coarse indicators and require physical verification. [1]

06 / CALCULATED RESPONSE

Know what the meter should show



Temperature (C)	NTC resistance (ohm)	Sense voltage (V)
15	15837	1.961
25	10000	1.600
35	6506	1.261
45	4348	0.970
55	2978	0.734

As temperature rises, NTC resistance falls and the measured voltage falls. This sign is a useful debugging check: increasing temperature with increasing voltage suggests the divider orientation or conversion equation is inconsistent.

Resistor substitution separates the electronics from thermal effects. Replace the NTC with a measured resistor and compute expected voltage using its measured value. A nominal 10k replacement gives 1.600 V, 6.8k gives 1.295 V, and 3.3k gives 0.794 V at a 3.20 V rail. These are calculated values, not recorded readings.

07 / UNCERTAINTY

Accuracy is an engineering budget

Illustrative contributor	At 25 C	At 55 C	Interpretation
ADC residual offset (10 mV)	0.28 C	0.48 C	Assumed scenario, not measured bound
Rail setting error (10 mV)	0.14 C	0.11 C	Assumed scenario, not measured bound
R1 tolerance, 0.1%	~0.02 C	~0.03 C	First-order sensitivity
R25 tolerance, 1%	~0.23 C	~0.27 C	Part-dependent, assumed
Beta tolerance, 1%	~0.00 C	~0.33 C	About 25 C reference
Self-heating / thermal gradients	Unknown	Unknown	Needs sensor data and bench test

At 25 C sensitivity is about -35.6 mV/C; at 55 C it is about -20.7 mV/C. The same ADC voltage error therefore causes more temperature error at the hot end. Do not sum these example contributors as a certified uncertainty or assume they are statistically independent.

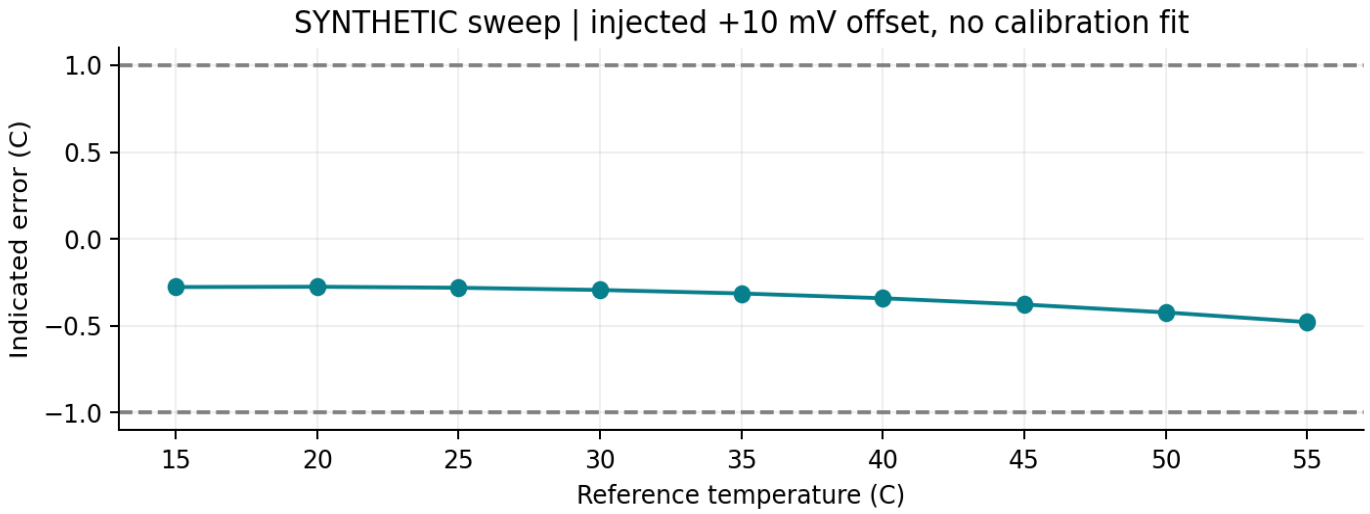
Calibration and verification are separate

First compare TP2 DMM voltage against logged mV at three resistor substitutions. Then use stable thermal points and an identified reference sensor. If correcting offset or slope, fit on one dataset and evaluate on separate points; retain raw readings and calibration coefficients.

Guardband example: if the justified expanded uncertainty of the comparison is U = 0.3 C, accept a +/-1 C requirement only when |observed error| + U <= 1 C. Reference uncertainty <=0.2 C alone does not prove U = 0.3 C; include gradients, repeatability and the complete measurement chain.

08 / VALIDATION PLOTS

A reproducible synthetic experiment



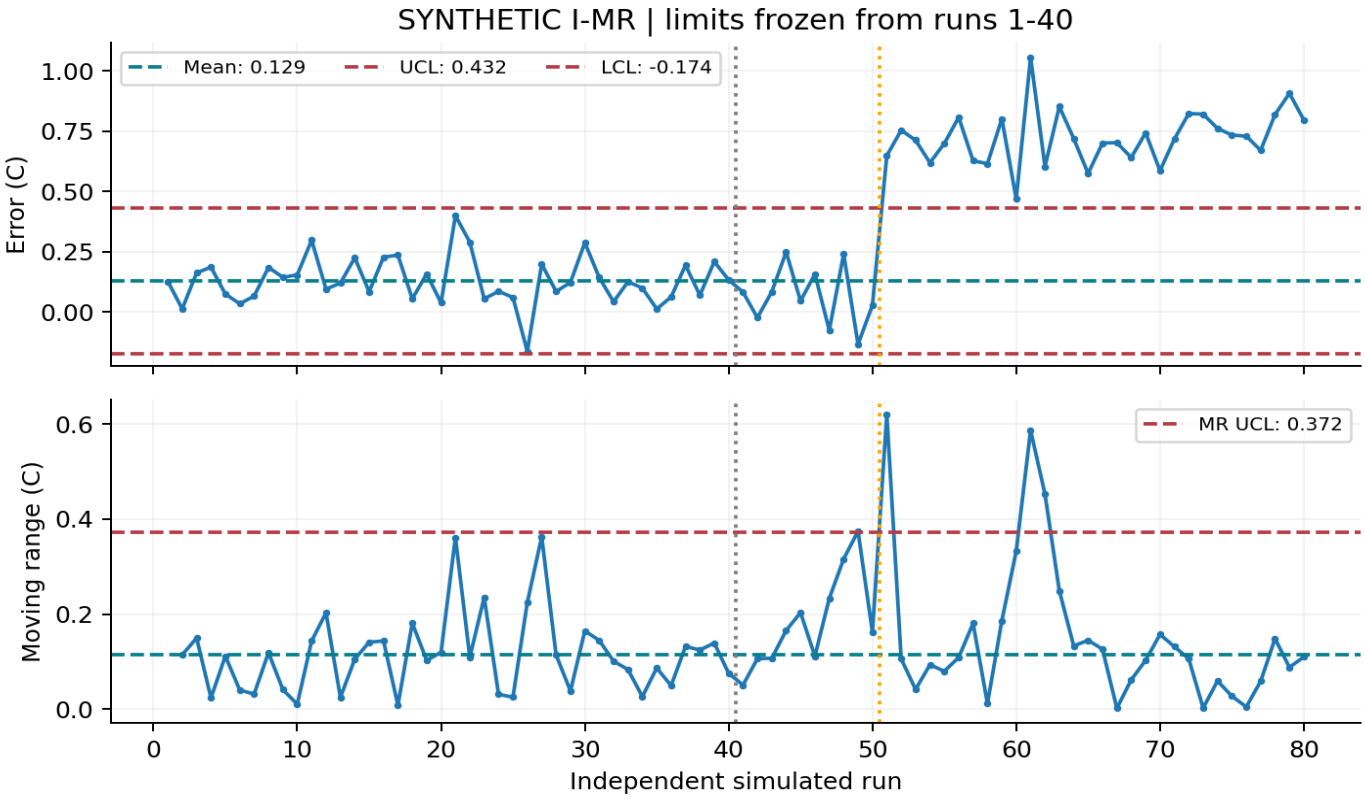
The curve uses the same nominal thermistor model, injects +10 mV at the ADC, and converts back to temperature. It demonstrates how a fixed voltage error produces a temperature-dependent error. It is not a calibration, a sensor accuracy result or an independent validation of the model.

Dataset	How it was produced	Proper use
synthetic_sweep.csv	Nine analytical temperatures; +10 mV voltage offset.	Discuss sensitivity and test planning.
synthetic_spc.csv	80 independent Gaussian errors; fixed RNG seed 20261019; +0.60 C shift from run 51.	Show monitoring calculations and drift detection.
hardware.csv	Created only by bench.py capture with connected hardware.	Actual bench evidence after provenance and setup are documented.

For a real thermal sweep, use stable plateaus rather than a moving heat source. Keep reference and NTC close together, allow settling, and record both. Do not immerse a bare unqualified sensor or breadboard; use a rated insulated probe and suitable setup if a bath is needed. Hand warming is a qualitative response demonstration only.

09 / SPC

Detect a change without hiding it



Baseline: runs 1-40. Mean error = **0.129 C**; mean moving range = **0.114 C**; estimated within sigma = MRbar/1.128 = **0.101 C**. I-chart limits: **-0.174 to 0.432 C**. MR UCL = 3.267 MRbar = **0.372 C**; lower limit is zero. [2]

Limits remain fixed after run 40. All 30 shifted points, runs 51-80, exceed the I-chart upper limit. The plotted rule is a single-point, three-sigma rule; it does not implement every run/trend rule. Investigate wiring, supply, sensor placement or calibration before assigning a physical cause.

Use independent stabilized runs for real SPC. Adjacent 5 Hz thermal readings are commonly correlated and can understate moving-range variation. This 40-run baseline is a teaching example, not proof of long-term stability. Control limits describe variation; specification limits express acceptability.

10 / CAPABILITY & EXECUTION

A test plan you can defend

For teaching only, using synthetic baseline errors and specifications -1 to +1 C: Cp = (USL-LSL)/(6 sigma) = **3.30**; Cpk = min(USL-mean, mean-LSL)/(3 sigma) = **2.88**. These high values were generated by an assumed noise distribution. They are **not capability evidence for this prototype**. Do not pool the shifted phase into a stable-process capability claim. [3]

Test / requirement	Procedure	Acceptance / present status
T1 / R1, R8	Inspect wiring; measure supply and TP1/TP2 using known resistors.	Match divider prediction within justified meter/resistor uncertainty. PENDING
T2 / R2	Five stable thermal points; record reference and DUT; repeat independently.	|error| + U <=1 C at all points. PENDING
T3 / R3	Fixed environment, after settling; capture 60 s; inspect correlation.	SD <=0.15 C; characterize drift separately. PENDING
T4 / R4, R7	Capture 60 s; inspect timestamps, gaps and parser counters.	4.5-5.5 Hz; zero malformed records. PENDING
T5 / R5	Disconnect/bypass NTC; restore each time.	Correct fault <=1 s; no valid temperature on rail fault. PENDING
T6 / R6	Controlled values equivalent to 46, 44, 42 C.	HOT, HOT, NORMAL; then 44 C remains NORMAL. PYTHON PASS; hardware pending
T7 / analysis	Run python -m unittest -v; rerun deterministic demo.	5/5 tests PASS in authoring runtime. Firmware and serial path untested.

Save board model, thermistor part number, measured R1 and supply, reference ID, ambient conditions, tool versions, raw CSV and code revision for each real run. A failed test is useful evidence when the root cause and next action are clear.

11 / SOFTWARE

What was used and what remains

Tool / language	Status for this package	Role
Python 3, NumPy, Matplotlib	Executed here	Model, seeded synthetic data, charts and statistics
unittest	Executed here; 5 tests passed	Nominal conversion, round trips, faults, hysteresis and range
ReportLab / Poppler	Used to create and inspect PDF	Document and vector schematic rendering
Arduino C++ / ESP32 core	Source supplied; not compiled	Sampling, conversion, hysteresis and serial output
pyserial	Capture code supplied; no connected board	Actual hardware CSV logging
Arduino IDE / VS Code	Suggested local tools; use not verified	Compile/upload and inspect Python code
LTspice / KiCad	Not used to produce these results	Optional future SPICE study / PCB capture; do not claim otherwise

Windows commands

```
py -m venv .venv
.\.venv\Scripts\python -m pip install -r requirements.txt
.\.venv\Scripts\python -m unittest -v
.\.venv\Scripts\python bench.py demo --out results
.\.venv\Scripts\python bench.py capture --port COM5 --seconds 60
  # Replace COM5 with your actual port; output defaults to hardware.csv
```

Place thermobench.ino inside a folder named thermobench. In Arduino IDE, install the Espressif ESP32 core, select the exact board and port, set measured constants, compile and upload. Close Serial Monitor before Python opens the same port. Record the actual core and library versions; this package does not claim a verified firmware toolchain.

Firmware uses analogReadMilliVolts rather than assuming 4095 codes equals 3.3 V. Espressif documents the calibrated-mV API; its availability does not establish system accuracy. Verify against the DMM. [1]

12 / INTERVIEW

Bring evidence, then tell the story

A truthful 60-second explanation

"This project is a temperature acquisition and validation bench. The design uses an NTC divider, an ESP32 ADC and a Python logger. I focused on making the signal traceable from voltage to temperature and defining how to test it. The report separates calculated results, synthetic examples and measurements. The supplied code was developed with AI assistance. I can explain the divider equations, alarm hysteresis and the validation plan, and I will distinguish what I have actually tested from what remains."

Show these in order

1. Point to the hardware and schematic; explain why voltage falls with heat.
2. Show a live nominal reading and compare voltage with the DMM.
3. Demonstrate gentle hand warming and an NTC disconnect fault.
4. Show a real CSV if you have captured one; otherwise label the synthetic example.
5. Explain the frozen SPC limits and one limitation you would address next.

Questions to practice

Question	Main answer
Why a 10k fixed resistor?	It matches the assumed 25 C NTC resistance and provides useful local sensitivity at modest current.
Why average 16 ADC readings?	Reduce random conversion noise; it does not fix gain error, nonlinearity or sensor tolerance.
Why 45/43 C thresholds?	Two-degree hysteresis prevents state chatter around a single threshold. These are demo design choices.
What does a high-rail fault mean?	Possible open NTC or low-side path, not a uniquely proven cause. Use DMM and wiring inspection.
What would you improve next?	Identify the actual NTC, verify ADC behavior, calibrate against a reference, then evaluate an external ADC if justified.

Before leaving: bring the powered working bench only if verified, protect the breadboard, save an offline CSV, charge the laptop, pack the USB data cable and print pages 1, 2, 4, 9 and 10. Also bring an original software sample you wrote and can walk through line by line.

13 / REFERENCES

Sources and release record

[1] Espressif Systems. Arduino-ESP32 ADC API. Consulted 19 October 2026.
docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html

[2] NIST/SEMATECH e-Handbook. Individuals Control Charts. Moving-range estimate and individual chart limits.
itl.nist.gov/div898/handbook/pmc/section3/pmc322.htm

[3] NIST/SEMATECH e-Handbook. What Is Process Capability? Stability is a prerequisite for meaningful capability interpretation.
itl.nist.gov/div898/handbook/pmc/section1/pmc16.htm

Interview scope source: user-supplied email screenshot from Jake Sakadjian requesting project prototypes, documentation (schematic, spec), and software examples. The screenshot does not identify the employer or prove the exact job description.

Release item	Rev A status
Python model / charts	Executed with deterministic random seed 20261019.
Python tests	5 passed; no hardware or C++ equivalence claim.
Firmware	Authored; compilation, upload, ADC behavior and timing unverified.
Schematic	Vector wiring diagram; no KiCad ERC, PCB layout or SPICE run.
Raw data	Synthetic datasets included; no fabricated hardware measurements.
Authorship	AI-assisted design, source and documentation prepared for review and learning.

Appendices contain the supplied firmware, Python analysis/logger and tests. The companion ZIP contains editable source, raw synthetic CSVs, figures and this PDF. Code snippets should be discussed only to the depth you can explain; describe your actual modifications and bench testing separately.

APPENDIX / COMPLETE SOURCE

thermobench.ino

AI-assisted source. Visual line wrapping only; use original file to run.

```
001 // AI-assisted interview prototype. Not compiled or bench-verified.
002 // Target: original ESP32-WROOM DevKit, ADC1 GPIO34.
003 #include <Arduino.h>
004 #include <math.h>
005 constexpr int PIN = 34;
006 // Replace with measured rail/resistor and identified NTC datasheet values.
007 constexpr float VS_MV=3200.0f, RF=10000.0f, R25=10000.0f, BETA=3950.0f;
008 const char* thermal="NORMAL";
009 uint32_t last=0;
010 void setup() {
011   Serial.begin(115200);
012   analogReadResolution(12);
013   analogSetPinAttenuation(PIN, ADC_11db);
014   Serial.println("ms,mv,temp_c,status");
015 }
016 void loop() {
017   if ((uint32_t)(millis()-last)<200) return;
018   last=millis();
019   float mv=0;
020   for(int i=0;i<16;i++) { mv+=analogReadMilliVolts(PIN); delay(1); }
021   mv/=16;
022   const char* status;
023   float t=NAN;
024   if(mv<=100) {status="LOW_FAULT"; thermal="NORMAL";}
025   else if(mv>=2900) {status="HIGH_FAULT"; thermal="NORMAL";}
026   else {
027     float r=RF*mv/(VS_MV-mv);
028     t=1.0f/(1.0f/298.15f+logf(r/R25)/BETA)-273.15f;
029     if(!isfinite(t) || t<15 || t>55) {
030       status="OUT_OF_RANGE"; thermal="NORMAL";
031     } else {
032       if(t>=45) thermal="HOT";
033       else if(t<=43) thermal="NORMAL";
034       status=thermal;
035     }
036   }
037   Serial.printf("%lu,%.2f,",(unsigned long)last,mv);
038   if(isfinite(t)) Serial.printf("%.3f",t);
039   Serial.printf(",%s\n",status);
040 }
```

APPENDIX / COMPLETE SOURCE

bench.py

AI-assisted source. Visual line wrapping only; use original file to run.

```
"""AI-assisted ThermoBench model, deterministic demo, SPC, serial logger.
Python 3 + numpy + matplotlib; pyserial only needed for capture.
All demo rows are SYNTHETIC. No hardware accuracy claim.
"""
from pathlib import Path
import argparse, csv, json, math
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
VS=3.2; RF=10000.; R25=10000.; BETA=3950.

def voltage(t):
    r=R25*np.exp(BETA*(1/(np.asarray(t)+273.15)-1/298.15))
    return VS*r/(RF+r)

def decode(v, previous='NORMAL'):
    if not math.isfinite(v): return None,'INVALID'
    if v<=.1: return None,'LOW_FAULT'
    if v>=2.9: return None,'HIGH_FAULT'
    r=RF*v/(VS-v)
    t=1/(1/298.15+math.log(r/R25)/BETA)-273.15
    if t<15 or t>55: return t,'OUT_OF_RANGE'
    return t, 'HOT' if t>=45 else ('NORMAL' if t<=43 else previous)

def capture(port,seconds,out):
    import serial, time
    valid={'NORMAL','HOT','LOW_FAULT','HIGH_FAULT','OUT_OF_RANGE'}
    with serial.Serial(port,115200,timeout=1) as s, open(out,'w',newline='') as f:
        w=csv.writer(f); w.writerow(['source','ms','mv','temp_c','status'])
        end=time.monotonic()+seconds; count=0; rejected=0
        while time.monotonic()<end:
            fields=s.readline().decode('ascii',errors='replace').strip().split(',')
            if fields==['ms','mv','temp_c','status']: continue
            try:
                if len(fields)!=4 or fields[3] not in valid: raise ValueError()
                ms=int(fields[0]); mv=float(fields[1])
                if ms<0 or not math.isfinite(mv): raise ValueError()
                if fields[2] and not math.isfinite(float(fields[2])): raise ValueError()
                if fields[3] in {'NORMAL','HOT','OUT_OF_RANGE'} and not fields[2]:
                    raise ValueError()
            except ValueError:
                rejected+=1; continue
            w.writerow(['HARDWARE',*fields]); f.flush(); count+=1
    print(json.dumps({'captured':count,'rejected':rejected,'file':out}))
    if count==0: raise SystemExit('No valid records received')

def demo(out):
    out=Path(out); out.mkdir(parents=True,exist_ok=True)
    rng=np.random.default_rng(20261019)
    # Independent demonstration errors, not ADC samples or actual chamber data.
    errors=rng.normal(.12,.10,80); errors[50:]+=.60
    baseline=errors[:40]; mr=np.abs(np.diff(baseline))
```

APPENDIX / COMPLETE SOURCE

bench.py

AI-assisted source. Visual line wrapping only; use original file to run.

```
    mean=float(baseline.mean()); mrbar=float(mr.mean()); sigma=mrbar/1.128
    lcl=mean-3*sigma; ucl=mean+3*sigma
    imr=np.abs(np.diff(errors)); mrucl=3.267*mrbar
    alarms=(np.flatnonzero((errors<lcl)|(errors>ucl))+1).tolist()
    cp=2/(6*sigma); cpk=min(1-mean,mean+1)/(3*sigma)
    metrics=dict(mean=mean,mrbar=mrbar,sigma=sigma,lcl=lcl,ucl=ucl,
                 mr_ucl=mrucl,cp_demo=cp,cpk_demo=cpk,i_alarms=alarms)
    (out/'metrics.json').write_text(json.dumps(metrics,indent=2))
    with (out/'synthetic_spc.csv').open('w',newline='') as f:

w=csv.writer(f);w.writerow(['source','run','reference_c','error_c','measured_c','phase'])
        for i,e in enumerate(errors):
            w.writerow(['SYNTHETIC',i+1,25,e,25+e,'BASELINE' if i<40 else 'MONITOR'])
    plt.rcParams.update({'font.size':10,'axes.spines.top':False,'axes.spines.right':False})
    t=np.linspace(15,55,401)
    fig,ax=plt.subplots(figsize=(8,3));ax.plot(t,voltage(t),color='#087f8c',lw=2)
    ax.set(xlabel='Temperature (C)',ylabel='Sense voltage (V)',title='Calculated transfer
curve | 3.20 V, 10k / B3950 model')

ax.grid(alpha=.2);fig.tight_layout();fig.savefig(out/'transfer.png',dpi=180);plt.close(fig)
    fig,axs=plt.subplots(2,1,figsize=(8,5),sharex=True)
    axs[0].plot(np.arange(1,81),errors,'.-',ms=4)
    for y,c,label in [(mean,'#087f8c','Mean'),(ucl,'#b53b45','UCL'),(lcl,'#b53b45','LCL')]:
        axs[0].axhline(y,color=c,ls='--',label=f'{label}: {y:.3f}')
    axs[0].set(ylabel='Error (C)',title='SYNTHETIC I-MR | limits frozen from runs 1-40')
    axs[0].legend(ncol=3,fontsize=8)
    axs[1].plot(np.arange(2,81),imr,'.-',ms=4)
    axs[1].axhline(mrucl,color='#b53b45',ls='--',label=f'MR UCL: {mrucl:.3f}')
    axs[1].axhline(mrbar,color='#087f8c',ls='--');axs[1].legend(fontsize=8)
    axs[1].set(xlabel='Independent simulated run',ylabel='Moving range (C)')
    for ax in axs:
ax.axvline(40.5,color='gray',ls=':');ax.axvline(50.5,color='orange',ls=':');ax.grid(alpha=.15)
    fig.tight_layout();fig.savefig(out/'spc.png',dpi=180);plt.close(fig)
    # Inject an offset in voltage, then show independent temperature points.
    refs=np.array([15.,20,25,30,35,40,45,50,55])
    measured=np.array([decode(float(voltage(x)+.010))[0] for x in refs])
    with (out/'synthetic_sweep.csv').open('w',newline='') as f:
        w=csv.writer(f);w.writerow(['source','reference_c','mv','measured_c','error_c'])
        for r,m in zip(refs,measured):
w.writerow(['SYNTHETIC',r,1000*(voltage(r)+.010),m,m-r])
    fig,ax=plt.subplots(figsize=(8,3));ax.plot(refs,measured-refs,'o-',color='#087f8c')
    ax.axhline(1,color='gray',ls='--');ax.axhline(-1,color='gray',ls='--')
    ax.set(xlabel='Reference temperature (C)',ylabel='Indicated error (C)',title='SYNTHETIC
sweep | injected +10 mV offset, no calibration fit')
    ax.grid(alpha=.2);fig.tight_layout();fig.savefig(out/'sweep.png',dpi=180);plt.close(fig)
    return metrics

if __name__=='__main__':
    p=argparse.ArgumentParser();sub=p.add_subparsers(dest='mode',required=True)
    d=sub.add_parser('demo');d.add_argument('--out',default='results')
    c=sub.add_parser('capture');c.add_argument('--port',required=True)
    c.add_argument('--seconds',type=int,default=60);c.add_argument('--
out',default='hardware.csv')
```

APPENDIX / COMPLETE SOURCE

bench.py

AI-assisted source. Visual line wrapping only; use original file to run.

```
100     a=p.parse_args()
101     if a.mode=='demo': print(json.dumps(demo(a.out),indent=2))
102     else: capture(a.port,a.seconds,a.out)
```

APPENDIX / COMPLETE SOURCE

test_bench.py

AI-assisted source. Visual line wrapping only; use original file to run.

```
001 import unittest
002 from bench import decode,voltage
003 class BenchTest(unittest.TestCase):
004     def test_nominal(self): self.assertAlmostEqual(decode(1.6)[0],25,places=8)
005     def test_roundtrip(self):
006         for t in [16,25,35,44,50,54]:
    self.assertAlmostEqual(decode(float(voltage(t)))[0],t,places=8)
007     def test_faults(self):
008         self.assertEqual(decode(0)[1],'LOW_FAULT')
009         self.assertEqual(decode(3.1)[1],'HIGH_FAULT')
010         self.assertEqual(decode(float('nan'))[1],'INVALID')
011     def test_hysteresis(self):
012         self.assertEqual(decode(float(voltage(46)))[1],'HOT')
013         self.assertEqual(decode(float(voltage(44)),'HOT')[1],'HOT')
014         self.assertEqual(decode(float(voltage(44)),'NORMAL')[1],'NORMAL')
015         self.assertEqual(decode(float(voltage(42)),'HOT')[1],'NORMAL')
016     def test_range(self): self.assertEqual(decode(float(voltage(60)))[1],'OUT_OF_RANGE')
017 if __name__=='__main__': unittest.main()
```